

Một lớp ứng dụng của hàm băm trong thi tin học

Yang Yi

2007

Nguồn gốc: A class of applications of hashing in informatics contests

IOI China candidate team papers / 2007 / day 2 / Yang Yi

Mở đầu

Bảng băm là một cấu trúc dữ liệu rất hiệu quả và có phạm vi ứng dụng rộng. Nếu hàm băm được thiết kế hợp lý, trong tình huống lý tưởng mỗi lần truy vấn chỉ tốn thời gian $\mathcal{O}(h/r)$, tức tỉ lệ với thương giữa dung lượng bảng băm và dung lượng còn trống. Chỉ cần dung lượng bảng băm lớn hơn khoảng 1.5 lần lượng dữ liệu thực sự được dùng, thời gian truy vấn về cơ bản có thể xem là hằng số $\mathcal{O}(1)$.

Với một bảng băm, hàm băm tốt đặc biệt quan trọng vì nó bảo đảm hiệu quả của bảng băm. Đặc trưng dễ thấy nhất của một hàm băm tốt là: những đối tượng khác nhau sau khi băm chỉ có xác suất rất nhỏ cho cùng một giá trị. Nhờ đó ta tránh được quá nhiều xung đột.

Bảng băm không thể tách khỏi hàm băm. Nhưng chiều ngược lại thì sao? Đôi khi hàm băm lại có thể được dùng độc lập với bảng băm. Một ví dụ thường gặp là thuật toán MD5 nổi tiếng: đó là một hàm băm, nhưng thường được dùng để mã hóa tóm tắt thông tin và kiểm tra tính đúng đắn của thông tin. Nói cách khác, ta trực tiếp so sánh kết quả do MD5 tính ra để suy đoán nội dung gốc có giống nhau hay không. Ngoài MD5, mã kiểm tra CRC32 và thuật toán SHA-1 thông dụng cũng dựa trên tư tưởng tương tự.

Vậy trong thi tin học, kiểu thuật toán này có đất dụng võ hay không?

Bài viết này bàn về loại hàm băm dùng với mục tiêu phát hiện trùng lặp hoặc phán định tương đương. Ta bắt đầu bằng Ví dụ 1.

1 Ví dụ 1: Ghép mẫu nhiều chiều

1.1 Mô tả bài toán

Tìm vị trí xuất hiện đầu tiên của một xâu trong một xâu khác là việc rất đơn giản: dùng KMP là đủ. Mở rộng lên hai chiều, bài toán trở thành tìm vị trí xuất hiện đầu tiên của một ma trận trong một ma trận khác. Nếu mở rộng lên k chiều thì phải làm thế nào?

Mảng văn bản X có kích thước theo các chiều là $N_1, N_2, \dots, N_k$, mảng mẫu Y có kích thước theo các chiều là $M_1, M_2, \dots, M_k$. Đặt

$$N = N_1 N_2 \cdots N_k, \quad M = M_1 M_2 \cdots M_k.$$

Bảo đảm $k \leq 10$, $N_i \geq M_i$, và cả N lẫn M đều không vượt quá 500000.

1.2 Phân tích thuật toán

Thuật toán thường gặp cho bài này là KMP nhiều chiều: trước hết tính tình trạng khớp ở một chiều, rồi xử lý hai chiều, ba chiều, ..., cho đến k chiều. Độ phức tạp thời gian là $\mathcal{O}(k(N + M))$. Tuy nhiên thuật toán này khó hiểu, khó nhớ, và không dễ hoàn thành cũng như viết đúng trong vài giờ thi.

Cách làm vết cạn tương đối dễ cài đặt, vì thế cũng rất dễ nghĩ ra; dữ liệu đối kháng cho nó lại rất dễ tạo. Do đó nó chỉ đáng thử khi hoàn toàn không có ý tưởng nào khác.

Có lựa chọn thứ ba hay không? Câu trả lời là có. Cách vết cạn tương đương với việc liệt kê mọi điểm bắt đầu, hiển nhiên số điểm bắt đầu không vượt quá N , rồi kiểm tra từng điểm. Nhưng kiểm tra hai mảng con có giống nhau hay không tốn tới $\mathcal{O}(M)$ thời gian. Đây chính là nguyên nhân khiến thuật toán vết cạn rất chậm trên dữ liệu đối kháng. Ta có thể loại nhanh một số trường hợp hiển nhiên không thể khớp trước khi so sánh hai mảng con hay không? Vì rất dễ tạo dữ liệu chỉ khác một ký tự, bất kể so sánh theo thứ tự nào cũng có thể tốn rất nhiều thời gian.

Hàm băm có thể phát huy tác dụng ở đây. Ta tính một giá trị băm cho mỗi mảng con có cùng kích thước với mảng mẫu. Rõ ràng chỉ khi giá trị băm của một mảng con bằng giá trị băm của mảng mẫu thì chúng mới đáng được so sánh.

KMP nhiều chiều bắt đầu từ trường hợp một chiều rồi mở rộng lên chiều cao hơn. Với thuật toán ghép mẫu bằng hàm băm, ta cũng có thể bắt đầu từ một chiều: đó chính là thuật toán Rabin–Karp.

Với một xâu S , có thể dùng hàm sau để tính giá trị băm:

$$f(S) = \sum_{i=1}^{\text{len}(S)} S[i] p^{\text{len}(S)-i} \pmod{q}.$$

Giá trị băm của xâu mẫu Y vì thế được tính rất dễ. Ký hiệu S_i là xâu con độ dài M của xâu văn bản X bắt đầu từ ký tự thứ i . Không khó thấy quan hệ giữa $f(S_{i+1})$ và $f(S_i)$:

$$\begin{aligned} f(S_i) &= \sum_{j=i}^{i+M-1} X[j] p^{i+M-1-j} \pmod{q}, \\ f(S_{i+1}) &= \sum_{j=i+1}^{i+M} X[j] p^{i+M-j} \pmod{q} \\ &= \left[p \sum_{j=i}^{i+M-1} X[j] p^{i+M-1-j} + X[i+M] - p^M X[i] \right] \pmod{q} \\ &= [pf(S_i) + X[i+M] - p^M X[i]] \pmod{q}. \end{aligned}$$

Nhìn từ góc độ khác, nếu bỏ qua phép lấy dư theo q , hàm này xem xâu như một số trong hệ cơ số p . Khi đó S_{i+1} thu được bằng cách “dịch trái” S_i một vị trí, bỏ ký tự đầu tiên rồi thêm ký tự mới ở bên phải. Vì vậy công thức truy hồi trên cũng là hiển nhiên.

Nhờ công thức truy hồi này, ta có thể tính mọi giá trị băm của các xâu con độ dài M trong X với thời gian tuyến tính.

Bây giờ mở rộng bài toán lên nhiều chiều. Số giá trị băm cần tính vẫn không vượt quá N ; vì thế chỉ cần có phương pháp truy hồi thích hợp, ta vẫn có thể tính mọi giá trị băm của các mảng con trong thời gian tuyến tính. Thật ra, một ma trận con M_1 hàng, M_2 cột có thể xem là một xâu gồm M_2 “dải dọc”. Ta trước hết tính giá trị băm cho mọi dải dọc độ dài M_1 , rồi dùng các giá trị đó để tính giá trị băm hai chiều.

Một điểm cần chú ý là giá trị p dùng để tính băm một chiều, tức băm của các dải dọc, và giá trị p dùng để tính băm hai chiều không được giống nhau. Nếu không, rất dễ nghĩ ra phản ví dụ sau:

$$\begin{bmatrix} a & b \\ a & a \end{bmatrix} \quad \text{và} \quad \begin{bmatrix} a & a \\ b & a \end{bmatrix}.$$

Hai ma trận này không giống nhau, nhưng kết quả băm chắc chắn bằng nhau nếu hai chiều dùng cùng cơ số. Điều đó trái với mục đích dùng băm. Vì vậy ở chiều thứ nhất và chiều thứ hai phải dùng các giá trị p khác nhau.

Trong trường hợp hai chiều, tính mọi dải dọc độ dài M_1 tốn $\mathcal{O}(N)$ thời gian; sau đó xem các giá trị băm của dải dọc như những “chữ cái” để tính băm theo chiều ngang, tức giá trị băm của các ma trận con, cũng tốn $\mathcal{O}(N)$ thời gian. Tổng độ phức tạp vẫn là $\mathcal{O}(N)$.

Hàm băm hai chiều là, với $\text{len}_1(S)$ và $\text{len}_2(S)$ lần lượt là kích thước của S theo hai chiều:

$$f_2(S) = \sum_{i_1=1}^{\text{len}_1(S)} \sum_{i_2=1}^{\text{len}_2(S)} p_1^{\text{len}_1(S)-i_1} p_2^{\text{len}_2(S)-i_2} S[i_1, i_2] \pmod{q}.$$

Ký hiệu X_{i_1, i_2} là ma trận con của X có góc trái trên tại hàng i_1 , cột i_2 , gồm M_1 hàng và M_2 cột. Khi đó

$$f_2(X_{i_1, i_2}) = \sum_{j_1=i_1}^{i_1+M_1-1} \sum_{j_2=i_2}^{i_2+M_2-1} p_1^{i_1+M_1-1-j_1} p_2^{i_2+M_2-1-j_2} X[j_1, j_2] \pmod{q}.$$

Trượt cửa sổ sang phải một cột, ta có

$$f_2(X_{i_1, i_2+1}) = \left[p_2 f_2(X_{i_1, i_2}) + \sum_{j_1=i_1}^{i_1+M_1-1} p_1^{i_1+M_1-1-j_1} X[j_1, i_2 + M_2] - p_2^{M_2} \sum_{j_1=i_1}^{i_1+M_1-1} p_1^{i_1+M_1-1-j_1} X[j_1, i_2] \right] \pmod{q}.$$

Hai tổng trong công thức lần lượt là giá trị băm một chiều của dải dọc mới đi vào và dải dọc cũ bị bỏ ra, vì thế chúng có thể được tính trước.

Mở rộng lên k chiều cũng tự nhiên như vậy. Trước hết tính mọi giá trị băm của mảng con kích thước $M_1 \times M_2 \times \dots \times M_{k-1}$, rồi xem những giá trị băm ấy như các ký tự và dùng băm Rabin–Karp để tính giá trị băm cho từng mảng con kích thước $M_1 \times M_2 \times \dots \times M_k$. Sau k lượt tính toán, ta thu được mọi giá trị băm cần thiết, với độ phức tạp thời gian $\mathcal{O}(kN + M)$.

Hàm băm ở k chiều có thể viết là

$$f_k(X_{i_1, i_2, \dots, i_k}) = \sum_{j_1=i_1}^{i_1+M_1-1} \sum_{j_2=i_2}^{i_2+M_2-1} \dots \sum_{j_k=i_k}^{i_k+M_k-1} \left(\prod_{t=1}^k p_t^{i_t+M_t-1-j_t} \right) \cdot X[j_1, j_2, \dots, j_k] \pmod{q}.$$

Khi trượt theo chiều thứ k , đặt

$$G(c) = \sum_{j_1=i_1}^{i_1+M_1-1} \dots \sum_{j_{k-1}=i_{k-1}}^{i_{k-1}+M_{k-1}-1} \left(\prod_{t=1}^{k-1} p_t^{i_t+M_t-1-j_t} \right) X[j_1, \dots, j_{k-1}, c] \pmod{q},$$

tức $G(c)$ là giá trị băm của lát $(k-1)$ chiều tại tọa độ c theo chiều cuối. Khi đó công thức truy hồi là

$$f_k(X_{i_1, \dots, i_k+1}) = [p_k f_k(X_{i_1, \dots, i_k}) + G(i_k + M_k) - p_k^{M_k} G(i_k)] \pmod{q}.$$

Nếu không có thao tác lấy dư theo q , hàm băm này có thể được hiểu như một phép chuyển cơ số. Khi mỗi p_t đều lớn hơn kích thước bảng chữ cái tương ứng, chỉ cần giá trị băm khác nhau là chắc chắn nội dung khác nhau. Nhưng trên thực tế giá trị tính ra sẽ lớn đến mức làm thuật toán mất ý nghĩa: tốc độ so sánh “giá trị băm” ấy không nhanh hơn so sánh trực tiếp. Vì vậy phép lấy dư là cần thiết.

Trong tình huống lý tưởng, nếu một hàm băm có thể sinh ra S giá trị khác nhau thì xác suất hai mảng con khác nhau cho cùng một giá trị chỉ là $1/S$. Hàm băm của ta dĩ nhiên chưa chắc đạt được lý tưởng ấy,

nhưng từ biểu thức $1/S$ ta dễ thấy rằng tăng S có thể giảm hiệu quả xác suất phán đoán sai. Từ đó còn rút ra một kết luận khác: nếu độ chính xác của một hàm băm chưa đủ, chỉ cần tính thêm một hàm băm khác với nó là có thể nâng cao đáng kể độ đúng. Dĩ nhiên trong bài này, độ phức tạp thời gian đã gần bằng

$$\mathcal{O}(kN + N \cdot (1/S) \cdot M),$$

vì hàm băm không phải là lý tưởng tuyệt đối. Chỉ cần S không nhỏ hơn đáng kể so với N/k , hạng sau sẽ không phải nút thắt, và không cần bỏ quá nhiều công sức vào hướng này. Ngược lại, tính nhiều hàm băm sẽ làm hằng số của thuật toán tăng rõ rệt.

Về cách chọn p và q , vì có thể hiểu quá trình này là chuyển sang cơ số p rồi lấy dư theo q , ta nên cố gắng tránh xung đột trong phạm vi cho phép. Có thể dùng các gợi ý sau:

1. p không nên quá nhỏ, nếu không rất dễ sai.
2. q có thể chọn là một số nguyên tố, sau đó chọn p sao cho với mọi i từ 1 đến $p-2$, ta có $p^i \bmod q \neq 1$.

Hàm băm này không chỉ có thể được tính bằng cách lần cửa sổ như trên, mà còn có thể tính và duy trì bằng chia đoạn, chia khối, cây đoạn, v.v. Một số cách cụ thể sẽ xuất hiện trong các ví dụ phía sau.

2 Ví dụ 2: Equal squares (Ural 1486)

2.1 Mô tả bài toán

Trong một ma trận ký tự $N \times M$, hãy tìm hai ma trận con hình vuông giống nhau, được phép giao nhau, sao cho cạnh của hai hình vuông tìm được là lớn nhất có thể.

2.2 Phân tích thuật toán

Sau kinh nghiệm ở Ví dụ 1, gặp bài này ta dễ nghĩ đến việc tìm kiếm nhị phân độ dài cạnh hình vuông, rồi dùng bảng băm để kiểm tra độ dài ấy có khả thi hay không. Hàm băm dùng giống trường hợp hai chiều ở Ví dụ 1. Độ phức tạp thời gian là $\mathcal{O}(NM \log(NM))$.

Nhưng đáng tiếc là giới hạn thời gian của bài này rất chặt; chương trình làm như vậy vẫn quá thời gian. Cuối cùng, sau khi dùng một sửa đổi có vẻ hơi mạo hiểm, chương trình mới qua được: trực tiếp kiểm tra xem có xuất hiện hai giá trị băm bằng nhau hay không. Nếu tồn tại hai giá trị băm bằng nhau thì coi độ dài cạnh đó là khả thi; nếu không thì coi là không khả thi.

Bài này đã được thông qua như vậy, nhưng sửa đổi ấy có quá mạo hiểm không? Thật ra, xác suất hai hình vuông con có nội dung khác nhau mà giá trị băm lại bằng nhau là rất nhỏ. Vì thế trong phần lớn trường hợp ta có thể xem rằng nếu hai giá trị băm bằng nhau thì nội dung gốc cũng bằng nhau.

Ở đây còn một vấn đề: bảng băm lưu cái gì? Nếu chỉ lưu kiểu Boolean mà không nén bit thì có vẻ tốn khá nhiều bộ nhớ; hơn nữa bảng băm không thể mở quá lớn, nên hai nội dung khác nhau vẫn có thể bị băm vào cùng một vị trí. Chỉ so sánh “vị trí này đã từng bị băm tới chưa” vẫn chưa đủ an toàn. Nếu một lần truy vấn có xác suất sai là một phần triệu, thì một triệu truy vấn đã tạo ra xác suất sai đáng kể.

Một mẹo nhỏ là tính hai giá trị băm: một giá trị dùng để xác định vị trí trong bảng băm, giá trị kia dùng để so sánh. Chỉ khi bắt đầu tìm ở một địa chỉ trong bảng băm và tìm thấy một phần tử có giá trị băm so sánh bằng với giá trị của mình, ta mới coi như phần tử ấy đã từng xuất hiện trong bảng.

3 Ví dụ 3: Ghép mẫu không ổn định

3.1 Mô tả bài toán

Cho hai chuỗi A và B . Mỗi lần có thể thực hiện một trong các thao tác sau:

- INSERT $i\ j$: chèn ký tự j vào trước ký tự thứ i của A .
- DELETE i : xóa ký tự thứ i của A .
- REVERSE $i\ j$: đảo ngược đoạn từ vị trí i đến vị trí j trong A .
- QUERY $i\ j$: hỏi độ dài lớn nhất có thể khớp giữa A bắt đầu từ ký tự thứ i và B bắt đầu từ ký tự thứ j , tức độ dài LCP của hậu tố của A bắt đầu tại i và hậu tố của B bắt đầu tại j .

3.2 Phân tích thuật toán

Nếu hai chuỗi không thay đổi, muốn tìm LCP chỉ cần xây dựng mảng hậu tố của $A + B$. Nhưng trong bài này chuỗi A liên tục thay đổi; hơn nữa từ các kiểu thay đổi có thể thấy mỗi thao tác đều khiến mảng hậu tố biến đổi rất lớn. Vì vậy ta nên tìm đường khác.

Với một k cho trước, không khó kiểm tra LCP của hai chuỗi có dài ít nhất k ký tự hay không: tính giá trị băm của k ký tự đầu tiên của hai chuỗi rồi so sánh chúng. Nếu bằng nhau, ta gần như có thể chắc chắn rằng LCP của hai chuỗi dài ít nhất k ; nếu khác nhau thì độ dài LCP chắc chắn nhỏ hơn k . Như vậy có thể tính LCP bằng tìm kiếm nhị phân.

Vấn đề còn lại chuyển thành cách tính giá trị băm của một chuỗi con. Ta vẫn dùng hàm băm Rabin–Karp. Nếu đã biết giá trị băm của S_1 và S_2 , không khó tính giá trị băm của $S_1 + S_2$:

$$\begin{aligned} f(S_1 + S_2) &= \sum_{i=1}^{\text{len}(S_1)+\text{len}(S_2)} p^{\text{len}(S_1)+\text{len}(S_2)-i} (S_1 + S_2)[i] \pmod{q} \\ &= \left[\sum_{i=1}^{\text{len}(S_1)} p^{\text{len}(S_1)+\text{len}(S_2)-i} S_1[i] + \sum_{i=1}^{\text{len}(S_2)} p^{\text{len}(S_2)-i} S_2[i] \right] \pmod{q} \\ &= [p^{\text{len}(S_2)} f(S_1) + f(S_2)] \pmod{q}. \end{aligned}$$

Các giá trị p^i hiển nhiên có thể được tiền xử lý trong $\mathcal{O}(n)$ thời gian. Vì vậy ta có thể dùng $\mathcal{O}(1)$ thời gian để ghép giá trị băm của hai chuỗi thành giá trị băm của chuỗi nối.

Do đó, có thể dùng danh sách liên kết theo khối để duy trì giá trị băm của các chuỗi con trong A . Cách làm tương tự như duy trì tổng từng phần, chỉ khác ở chỗ giá trị băm xuôi và ngược của một chuỗi là khác nhau. Để có thể hoàn thành thao tác reverse trong $\mathcal{O}(\sqrt{n})$ thời gian, cần đảo ngược một khối trong $\mathcal{O}(1)$ thời gian; vì thế mỗi khối phải duy trì hai giá trị băm: một theo chiều xuôi và một theo chiều ngược. Ngoài điểm này, các thao tác còn lại tương tự như duy trì tổng từng phần hoặc duy trì giá trị lớn nhất. Khi đó thao tác chèn, xóa, đảo ngược đều tốn $\mathcal{O}(\sqrt{n})$, còn truy vấn tốn $\mathcal{O}(\sqrt{n} \log n)$.

Tương tự, một cách giải khác là duy trì giá trị băm trên cây splay. Mỗi khi một nút được xoay hoặc cây con gốc tại nút ấy bị sửa đổi, ta tính lại giá trị băm xuôi và băm ngược của nó. Nhờ vậy có thể đảo ngược một cây con trong $\mathcal{O}(1)$ thời gian; thông qua thao tác tách, có thể đảo ngược một cây con trong thời gian khấu hao $\mathcal{O}(\log n)$. Thao tác chèn và xóa hiển nhiên cũng có thời gian khấu hao $\mathcal{O}(\log n)$, còn truy vấn có độ phức tạp khấu hao $\mathcal{O}(\log^2 n)$.

4 Ví dụ 4: Một lớp bài toán phán định đẳng cấu

4.1 Bài toán 1: so sánh hai cây có gốc

Thuật toán dễ nghĩ là dùng hai xâu để biểu diễn hai cây. Nhưng nếu dùng băm thì nên làm thế nào?

Ta có thể dùng một kiểu truy hồi trên cây để tính giá trị băm. Với một nút v , trước hết tính giá trị băm của mọi con của nó, rồi sắp xếp các giá trị đó tăng dần, ký hiệu là $H_1, H_2, \dots, H_D$. Khi đó giá trị băm của v có thể được tính như sau:

$$\text{Hash}(v) = (((\dots (((a \cdot p \text{ xor } H_1) \bmod q) \cdot p \text{ xor } H_2) \bmod q) \dots \cdot p \text{ xor } H_D) \cdot b) \bmod q).$$

Nói cách khác, ta bắt đầu từ một hằng số, mỗi lần nhân với p , xor với một phần tử, lấy dư theo q , rồi tiếp tục nhân với p , xor với phần tử kế tiếp, lại lấy dư theo q , ..., cho đến phần tử cuối. Cuối cùng nhân kết quả với b rồi lấy dư theo q .

Lý do phải sắp xếp trước giá trị băm của các con là: chỉ khác thứ tự con không làm hai cây khác nhau. Còn cách băm phía sau có thể tương đối tùy ý; không nhất thiết phải dùng đúng hàm băm này, miễn là chọn một hàm băm đủ tốt. Nhưng cần chú ý rằng hàm Rabin–Karp ở trên không thích hợp để dùng trực tiếp tại đây, vì không khó tìm phản ví dụ.

Hãy xét hai cây sau bằng mô tả xâu ngoặc: cây trái có gốc với hai con, một con là lá và một con có một lá; cây phải có gốc với hai con, một con có hai lá và một con là lá. Nếu trong cây phải thứ tự con đúng như mô tả, thì vì giá trị băm có thể xem là phân bố ngẫu nhiên nên có một nửa khả năng thứ tự này xuất hiện sau khi sắp xếp. Giá trị băm của mọi lá bắt buộc giống nhau, ký hiệu là l . Nếu dùng truy hồi

$$\text{Hash}(v) = b \sum_{i=1}^D p^{D-i} v_i \pmod{q},$$

thì cây trái có giá trị

$$lb^2(p^2 + p + 1)(p + 1) \pmod{q},$$

còn cây phải có giá trị

$$lb^2[(p + 1)p + (p^3 + p^2 + p + 1)] \pmod{q}.$$

Hai giá trị này bằng nhau, nhưng hai cây lại không bằng nhau.

Tương tự, với cách tính ở phần đầu, nếu cuối cùng không nhân với b thì cũng sai; có thể phân tích trường hợp cây suy biến thành một đường thẳng.

Vậy, nếu so sánh trực tiếp giá trị băm chắc chắn có xác suất sai, vì sao còn cần chỉ ra hàm băm nào dùng được, hàm nào không dùng được? Một số lỗi là “sai số ngẫu nhiên”: không cần đổi phương pháp tính băm, chỉ đổi các hằng như p là có thể loại bỏ. Nhưng có những lỗi là “sai số hệ thống”, do bản thân hàm băm không hợp lý; trường hợp sau nên tránh tối đa. Dĩ nhiên trong vài giờ thi không nhất thiết ta đánh giá được một hàm băm có hợp lý hay không, nhưng tốt hơn nên chọn hàm băm mà khi chưa biết các hằng có thể thay đổi như p, q , rất khó dựng phản ví dụ.

Bây giờ chỉ cần so sánh giá trị băm của hai cây là có thể phân biệt chúng có giống nhau hay không. Độ phức tạp thời gian là $\mathcal{O}(n \log n)$.

Còn một hàm băm khác cũng giải được bài toán này, và rất hữu ích cho bài toán tiếp theo. Ta xét việc biểu diễn cây bằng một xâu, tương tự biểu diễn nhỏ nhất, rồi tính giá trị băm của xâu ấy. Tất nhiên nếu thật sự dựng toàn bộ xâu như phương pháp biểu diễn nhỏ nhất thì hơi không đáng. Ta chỉ cần dựa vào giá trị băm của các con để sắp xếp các con. Vì hàm băm Rabin–Karp xử lý được giá trị băm của phép nối hai xâu, sau khi đã xác định thứ tự và biết số nút của từng con, tức độ dài xâu con, không khó xác định giá trị băm của nút hiện tại.

Cụ thể, nếu $c_1, c_2, \dots, c_D$ là các con của nút hiện tại v , và $s(x)$ biểu diễn xâu chuyển đổi từ cây con gốc x , thì

$$s(v) = g(v)s(c_1)s(c_2) \cdots s(c_D).$$

Ở đây $g(v)$ là một hàm nào đó có năng lực phân biệt nhất định về v , chẳng hạn số con của v , hoặc một hàm băm nào đó liên quan tới v .

Với hàm băm Rabin–Karp

$$f(S) = \sum_{i=1}^{\text{len}(S)} S[i]p^{\text{len}(S)-i} \pmod{q},$$

ta có

$$\begin{aligned} f(s(v)) &= f(g(v)s(c_1)s(c_2) \cdots s(c_D)) \\ &= \left[p^{\text{len}(s(v))-1} g(v) + \sum_{i=1}^D f(s(c_i)) p^{\sum_{j=i+1}^D \text{len}(s(c_j))} \right] \pmod{q}. \end{aligned}$$

Vì vậy chỉ cần một lần truy hồi trên cây là có thể tính giá trị băm.

4.2 Bài toán 2: so sánh hai cây vô căn

Lúc này nếu tiếp tục dùng biểu diễn nhỏ nhất thì độ phức tạp rất cao. Nhưng nếu dùng băm hợp lý, độ phức tạp vẫn chỉ là $\mathcal{O}(n \log n)$.

Cách làm cụ thể như sau. Với mỗi cạnh $(a, b) \in E$, tính một giá trị băm $F(a, b)$ sao cho nó phụ thuộc vào mọi $F(b, i)$ thỏa $i \neq a$ và $(b, i) \in E$. Ở đây $F(a, b)$ và $F(b, a)$ là hai khái niệm khác nhau. Giá trị $F(a, b)$ dùng để biểu diễn “khi xem a là cha của b , giá trị băm của cây con gốc b ”. Để thấy cách truy hồi này không gây chu trình và chắc chắn có thể tính theo một thứ tự nào đó.

Trước hết chọn tùy ý một nút r làm gốc. Với một cặp có thứ tự (a, b) thỏa $(a, b) \in E$, nếu a gần gốc hơn b thì gọi (a, b) là hướng xuống, ngược lại là hướng lên. Rõ ràng có thể dùng phương pháp thứ hai trong bài toán 1 để tính mọi $F(a, b)$ hướng xuống trong $\mathcal{O}(n \log n)$ thời gian.

Bây giờ xét gốc r . Mọi (r, i) đều là hướng xuống nên mọi $F(r, i)$ đều đã biết. Ta cần tính các giá trị $F(i, r)$. Với một con t của r , giá trị $F(t, r)$ có thể được tính từ giá trị băm của mọi con khác của r ngoài t . Nếu tính trực tiếp như vậy, trường hợp xấu nhất sẽ tốn $\mathcal{O}(n^2)$.

Ở đây dùng một mẹo nhỏ trong truy hồi trên cây. Vì mọi $F(r, i)$ đều đã biết, ta sắp xếp chúng trước rồi xét. Hàm băm này về bản chất vẫn là băm của xâu; ta sắp các điểm kề với r theo $F(r, i)$, ký hiệu lần lượt là $c_1, c_2, \dots, c_D$, và ký hiệu $s(a, b)$ là xâu tương ứng với $F(a, b)$.

Có thể tìm kiếm nhị phân vị trí k của t trong $c_1, c_2, \dots, c_D$. Khi đó

$$s(t, r) = g(r)s(r, c_1)s(r, c_2) \cdots s(r, c_{k-1})s(r, c_{k+1}) \cdots s(r, c_D).$$

Các phần $g(r), s(r, c_1)s(r, c_2) \cdots s(r, c_{k-1})$ và $s(r, c_{k+1}) \cdots s(r, c_D)$ sau khi sắp xếp đều có thể được tiền xử lý trong $\mathcal{O}(n)$ thời gian tương tự Ví dụ 1. Vì vậy có thể tính $F(t, r)$ trong $\mathcal{O}(\log n)$ thời gian, do phải tìm kiếm nhị phân.

Khi một nút v đã có $F(v, \text{father}(v))$, thì tương đương với việc mọi $F(v, i)$ với i kề v đều đã biết. Tình huống này giống hệt thảo luận vừa rồi về gốc r , nên không cần lặp lại chi tiết.

Như vậy, lượt DFS thứ nhất xác định mọi $F(a, b)$ hướng xuống; lượt DFS thứ hai xác định mọi $F(a, b)$ hướng lên. Độ phức tạp thời gian là bao nhiêu? Lượt đầu hiển nhiên là $\mathcal{O}(n \log n)$. Ở lượt hai, với một nút có D con, chi phí là $\mathcal{O}(D \log D) < \mathcal{O}(D \log n)$; vì $\sum D = N$, tổng thời gian vẫn là $\mathcal{O}(n \log n)$.

Giá trị băm của một đỉnh có thể được định nghĩa là giá trị băm của cả cây khi lấy đỉnh ấy làm gốc. Sau khi đã tính mọi $F(a, b)$, giá trị băm của từng đỉnh được tính không khó.

Cuối cùng, nếu hai cây vô căn có thể ghép từng đôi giá trị băm của đỉnh với nhau, tức sau khi sắp xếp thì hoàn toàn tương ứng, hoặc kiểm tra bằng một bảng băm, ta có thể coi chúng bằng nhau; nếu không thì coi là không bằng nhau.

Vì giá trị băm của một đỉnh tương đương với việc lấy đỉnh ấy làm gốc, thu được một cây có gốc rồi băm cây đó, có thể trực tiếp dùng giá trị băm đỉnh để so sánh hai đỉnh có ở vị trí tương đương trong cây hay không. Cũng có thể dựa trên sự tương ứng từng đỉnh để thu được ánh xạ giữa các đỉnh của hai cây có cùng cấu trúc nhưng đánh số khác nhau. Do thay đổi một đỉnh sẽ làm giá trị băm của toàn bộ các nút trên cây thay đổi, việc hai giá trị băm bằng nhau cũng phản ánh rằng “hai nút này rất có thể thuộc cùng một cây, hoặc thuộc hai cây hoàn toàn giống nhau”. Kiểu băm này thậm chí có thể dùng để phán định những điểm nào trong một rừng ở vị trí tương đương.

4.3 Bài toán 3: so sánh hai đồ thị có hướng

Vì trường hợp đồ thị vô hướng có thể chuyển thành đồ thị có hướng để xét, ở đây chỉ bàn về đồ thị có hướng. Tác giả chưa tìm được một phương pháp băm thật sự hiệu quả, nhưng có một thuật toán trong thực tế chưa gặp vấn đề.

Trước hết liệt kê một điểm bắt đầu i , đặt giá trị băm của mọi nút thành một hằng số, chẳng hạn 1. Sau đó lặp: mỗi lần giá trị băm mới của một nút được tính là

$$f_{j+1}(v) = \left[af_j(v) + b \sum_{\substack{w \in V \\ v \rightarrow w \in E}} f_j(w) + c \sum_{\substack{w \in V \\ w \rightarrow v \in E}} f_j(w) + g(v) \right] \bmod p,$$

trong đó

$$g(v) = \begin{cases} d, & v = i, \\ 0, & v \neq i. \end{cases}$$

Sau khi lặp k lần, giá trị $f_k(i)$ thu được được xem là giá trị băm của điểm i . Mấu chốt là phép tính $f_j(i)$ khác các giá trị khác, nhờ đó tránh được nhiều trường hợp đặc biệt.

Bài liên quan: The labyrinth of wells (POI 1999/2000 Stage 2). Tóm tắt bài toán: có nhiều phòng; phòng trên cùng là điểm xuất phát, phòng dưới cùng là điểm kết thúc. Trừ phòng dưới cùng, mỗi phòng có một trong ba màu và có ba ống khác nhau dẫn tới các phòng bên dưới nó. Mọi phòng đều có thể tới được, và mọi ống cuối cùng đều dẫn tới phòng dưới cùng. Nhưng một số nút về bản chất là giống nhau: nếu chọn cùng một cách đi thì sẽ đi qua cùng chuỗi màu. Hỏi sau khi gộp các nút bản chất giống nhau, số điểm còn lại ít nhất là bao nhiêu.

Hai điểm có thể gộp khi và chỉ khi các đường đi phía sau chúng có màu hoàn toàn giống nhau. Từ đó có thể thiết kế một hàm băm:

$$\text{Hash}(x) = f(\text{Hash}(x \rightarrow c_1), \text{Hash}(x \rightarrow c_2), \text{Hash}(x \rightarrow c_3), x \rightarrow \text{color}),$$

trong đó f là một hàm băm nào đó. Trong chương trình của tác giả, hàm băm là

$$f(a, b, c, d) = ((129(129a \text{ xor } b) \text{ xor } c) \text{ xor } 987654321) + d.$$

Nhân tiện, để tăng hiệu suất chương trình, có thể dùng một số kỹ thuật thao tác bit, chẳng hạn $a \cdot 129 = (a \ll 7) + a$.

5 Ví dụ 5: Biểu thức tương đương (NOIP 2005)

5.1 Mô tả bài toán

Cho một đa thức chưa rút gọn, rồi cho tiếp n đa thức. Hãy xác định trong n đa thức ấy, những đa thức nào bằng đa thức ban đầu.

Ràng buộc: $n \leq 26$, độ dài biểu thức không vượt quá 50. Trong đa thức chỉ có số, biến a , các phép $+$, $-$, $*$, $^$ (lũy thừa), dấu ngoặc trái phải và khoảng trắng. Các số đều nhỏ hơn 10000, số mũ trong phép lũy thừa không vượt quá 10; $+$, $-$, $*$, $^$, $($, $)$ đều chỉ đóng vai trò toán tử, và không có trường hợp lược bỏ dấu nhân.

5.2 Phân tích thuật toán

Cách làm thông thường là khai triển từng đa thức, gộp các hạng tử đồng dạng, rồi so sánh. Thuật toán này tất nhiên có thể qua dữ liệu kiểm tra khi thi.

Nhưng với dữ liệu như sau thì sao? Hiển nhiên dữ liệu này vẫn phù hợp yêu cầu đề bài:

$$(a + 9999)^{9999}.$$

Không khó thấy rằng đa thức sau khi khai triển căn bản không thể lưu được, và các hệ số phía sau cũng khó biểu diễn. Trước một đối tượng khổng lồ như vậy, làm sao so sánh hai đa thức đều lớn đến thế có bằng nhau hay không? Rõ ràng ngay cả tính toàn bộ biểu thức ra đã cực kỳ khó, chưa nói đến so sánh.

Thật ra cách giải rất đơn giản: thay một hoặc vài giá trị vào biến a , rồi dựa vào phần dư của kết quả sau khi chia cho một số lớn để trực tiếp phán định hai đa thức có bằng nhau hay không. Nếu không lấy dư, dữ liệu “khổng lồ” ở trên vẫn không xử lý được. Việc thay một giá trị vào rồi tính kết quả cũng có thể xem là áp dụng một hàm băm đặc biệt cho hai đa thức, sau đó so sánh giá trị băm để so sánh biểu thức gốc. Dĩ nhiên, nếu chọn vài giá trị phù hợp cho a , có thể chứng minh thuật toán này trở thành chắc chắn đúng.

6 Ví dụ 6: Guess the Number

6.1 Mô tả bài toán

Bài này được cải biên từ OIBH Reminiscence Programming Contest. Cho một số S có tối đa 500000 chữ số. Hãy tìm N sao cho $N^N = S$. Tuy nhiên, số S được cho có thể có một vài chữ số sai, nhưng không có chuyện thêm hoặc bớt chữ số; khi đó cần in ra -1 .

6.2 Phân tích thuật toán

Trước hết, với S nhỏ có thể kiểm tra trực tiếp. Còn với $S > 10^{10}$, ta có thể xác định một giá trị N khả dĩ, vì khi $N \geq 10$ thì

$$\frac{(N+1)^{N+1}}{N^N} > 10,$$

nên hai lũy thừa liên tiếp không thể có cùng số chữ số. Phần còn lại là kiểm tra N ấy có thỏa $N^N = S$ hay không.

Hiển nhiên tính logarit là không thể dựa vào, vì kiểu số thực không có độ chính xác cao như vậy. Tương tự, tính số nguyên lớn cũng khó thực hiện; ngay cả FFT cũng gặp thách thức về độ phức tạp thời

gian. Liên hệ với các ví dụ trước, ta có thể nghĩ đến việc tìm một hàm băm $f(x)$, rồi so sánh $f(N^N)$ và $f(S)$ để xác định N^N có bằng S hay không. Đồng thời, hàm $f(x)$ còn cần thỏa: nếu $x = ab$, ta có thể dễ dàng tính $f(x)$ từ $f(a)$ và $f(b)$, nhờ đó so sánh trực tiếp mà không cần tính ra N^N .

Một hàm băm rất đơn giản nhưng hiệu quả là lấy dư:

$$f(x) = x \bmod p.$$

Nếu $x = ab$ thì

$$f(x) = f(a)f(b) \bmod p.$$

Với bài gốc, trong đó S và N^N nhiều nhất chỉ khác một chữ số, chỉ cần p không chia hết cho 10 là có thể bảo đảm phán định chính xác.

Với phiên bản cải biên, nơi S có thể sửa tùy ý nhiều chữ số, thì sao? Lấy một vài giá trị p . Do giới hạn độ dài của S chỉ là 500000 chữ số, tích các số nguyên tố nhỏ hơn 130000 đã vượt quá phạm vi có thể chia hết bởi $|S - N^N|$. Nói cách khác, nếu chọn ngẫu nhiên một p trong phạm vi 500000, xác suất p vừa khéo chia hết $|S - N^N|$ chỉ khoảng $1/4$. Nếu chọn nhiều giá trị hơn, hoặc mở rộng phạm vi của p , ta gần như chắc chắn phán định đúng.

Tổng kết

Ngoài vai trò công cụ phụ trợ cho bảng băm, khi dùng độc lập, hàm băm có thể phán định gần như chắc chắn hai dữ liệu có giống nhau hoặc tương đương hay không. Bản chất của hàm băm là “tóm lược” thông tin có lượng dữ liệu lớn. Vì vậy khi gặp bài toán cần thường xuyên so sánh hai dữ liệu có giống nhau hay không, ta nên cân nhắc dùng hàm băm như một công cụ giải bài.

Đôi khi dùng hàm băm sẽ đánh đổi một phần tính đúng đắn, và thuật toán dùng hàm băm nhìn qua cũng không đủ đẹp. Nhưng nếu dùng hàm băm linh hoạt, ta có thể biến khó thành dễ, giải những bài vốn khó xử lý, và thường đạt hiệu quả cao bằng lượng mã ngắn.

Dù có điều phải bỏ, cũng có điều nhận lại; chính vì có điều bỏ đi nên mới có điều nhận được. Khi chọn thuật toán, ta luôn gặp kiểu mâu thuẫn như vậy: có mất thì có được, có được thì cũng có mất. Thuật toán tốt nhất là thuật toán phù hợp với đề bài và cũng phù hợp với thói quen tư duy của mình.

Tài liệu tham khảo

- Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.
- Chen Qifeng, *NEW LCP*, bài tập đội tuyển tập huấn quốc gia năm 2006.
- *Cấu trúc dữ liệu*, Nhà xuất bản Đại học Thanh Hoa.